JavaScript Source Code

```
// ---------- Data (saved in the browser's IndexedDB) ----------
const DB_NAME = "library_db";
const BOOKS_KEY = "lms_books";   // old localStorage keys, only used for a one-time import
const ISSUES_KEY = "lms_issues";
let db = null;                   // the open IndexedDB connection

let books = [];
let issues = []; // each: { id, bookId, student, issueDate, dueDate, returned }
let editingId = null; // Book ID being edited, or null when adding

// Dates use the computer's local time (toISOString gives the UTC date, which can be a day off in India)
function formatDate(d) {
  const month = String(d.getMonth() + 1).padStart(2, "0");
  const day = String(d.getDate()).padStart(2, "0");
  return `${d.getFullYear()}-${month}-${day}`;
}

function today(offsetDays = 0) {
  const d = new Date();
  d.setDate(d.getDate() + offsetDays);
  return formatDate(d);
}

// Adds days to a "YYYY-MM-DD" string and returns a "YYYY-MM-DD" string
function addDays(dateString, days) {
  const [y, m, d] = dateString.split("-").map(Number);
  return formatDate(new Date(y, m - 1, d + days));
}

// Shows "2026-10-18" as "18 Oct 2026"
function showDate(dateString) {
  if (!dateString) return "-";
  const [y, m, d] = dateString.split("-").map(Number);
  return new Date(y, m - 1, d).toLocaleDateString("en-GB", { day: "numeric", month: "short", year: "numeric"
});
}

// Opens (and on first use, creates) the database with three object stores
function openDatabase() {
  return new Promise((resolve, reject) => {
    const request = indexedDB.open(DB_NAME, 1);
    request.onupgradeneeded = () => {
      request.result.createObjectStore("books", { keyPath: "id" });
      request.result.createObjectStore("issues", { keyPath: "id" });
      request.result.createObjectStore("meta", { keyPath: "key" });
    };
    request.onsuccess = () => { db = request.result; resolve(); };
    request.onerror = () => reject(request.error);
  });
}

// Runs one request in its own transaction and resolves when it has finished
function dbRun(storeName, mode, action) {
  return new Promise((resolve, reject) => {
    const tx = db.transaction(storeName, mode);
    const request = action(tx.objectStore(storeName));
    tx.oncomplete = () => resolve(request.result);
    tx.onerror = () => reject(tx.error);
    tx.onabort = () => reject(tx.error);
  });
}
const dbGetAll = store => dbRun(store, "readonly", s => s.getAll());
const dbPut = (store, value) => dbRun(store, "readwrite", s => s.put(value));
const dbDelete = (store, key) => dbRun(store, "readwrite", s => s.delete(key));

// Loads the latest books and issue records from the database
async function refresh() {
  // IndexedDB returns books sorted by ID, so "addedAt" restores the order they were added in
  books = (await dbGetAll("books")).sort((a, b) => a.addedAt - b.addedAt);
  issues = await dbGetAll("issues");
}

function saveFailed(err) {
  console.error(err);
  showMessage("Could not save to the browser database. Please try again.", "error");
}

// Books and issues saved by the earlier localStorage version (or null if there are none)
function readOldData() {
```

```
  try {
    const oldBooks = JSON.parse(localStorage.getItem(BOOKS_KEY));
    if (!Array.isArray(oldBooks)) return null;
    const oldIssues = JSON.parse(localStorage.getItem(ISSUES_KEY)) || [];
    oldIssues.forEach(i => {
      if (!i.bookTitle) {
        const b = oldBooks.find(x => x.id === i.bookId);
        i.bookTitle = b ? b.title : "(deleted book)";
      }
      if (i.returnDate === undefined) i.returnDate = "";
    });
    return { books: oldBooks, issues: oldIssues };
  } catch (e) {
    return null;
  }
}

function sampleData() {
  return {
    books: [
      { id: "B-101", title: "To Kill a Mockingbird", author: "Harper Lee", category: "Fiction", quantity: 3 },
      { id: "B-102", title: "A Brief History of Time", author: "Stephen Hawking", category: "Science",
quantity: 2 },
      { id: "B-103", title: "Clean Code", author: "Robert C. Martin", category: "Technology", quantity: 4 },
      { id: "B-104", title: "Sapiens", author: "Yuval Noah Harari", category: "History", quantity: 2 },
      { id: "B-105", title: "The Alchemist", author: "Paulo Coelho", category: "Fiction", quantity: 1 }
    ],
    issues: [
      { id: 1, bookId: "B-105", bookTitle: "The Alchemist", student: "Aarav Mehta", issueDate: today(-3),
dueDate: today(11), returned: false, returnDate: "" }
    ]
  };
}

// First visit only: import old localStorage data if it exists, otherwise add the sample books
async function setUpData() {
  const done = await dbRun("meta", "readonly", s => s.get("seeded"));
  if (done) return;
  const start = readOldData() || sampleData();
  await new Promise((resolve, reject) => {
    const tx = db.transaction(["books", "issues", "meta"], "readwrite");
    start.books.forEach((b, i) => tx.objectStore("books").put({ ...b, addedAt: i }));
    start.issues.forEach(i => tx.objectStore("issues").put(i));
    tx.objectStore("meta").put({ key: "seeded", value: true });
    tx.oncomplete = () => resolve();
    tx.onerror = () => reject(tx.error);
    tx.onabort = () => reject(tx.error);
  });
}

// ---------- Helpers ----------
function $(id) { return document.getElementById(id); }

// Prevents typed text from being treated as HTML
function esc(text) {
  const div = document.createElement("div");
  div.textContent = text;
  return div.innerHTML;
}

function findBook(id) { return books.find(b => b.id === id); }
function activeIssues() { return issues.filter(i => !i.returned); }
function issuedCount(bookId) { return activeIssues().filter(i => i.bookId === bookId).length; }
function availableCount(book) { return book.quantity - issuedCount(book.id); }

// Text and color class for a book's availability, e.g. "2 of 3 available"
function availabilityInfo(book) {
  const avail = availableCount(book);
  let cls = "available";
  let status = "Available";
  if (avail === 0) { cls = "none"; status = "All issued"; }
  else if (avail < book.quantity) { cls = "partial"; status = "Partly issued"; }
  return { cls, status, text: `${avail} of ${book.quantity} available` };
}

let messageTimer;
function showMessage(text, type) {
  const box = $("message");
  box.textContent = text;
  box.className = "message " + type;
  clearTimeout(messageTimer);
  messageTimer = setTimeout(() => box.classList.add("hidden"), 3500);
}
```

```
function emptyRow(cols, text) {
  return `<tr><td class="empty" colspan="${cols}">${text}</td></tr>`;
}

// ---------- Navigation ----------
function showPage(name) {
  document.querySelectorAll(".page").forEach(p => p.classList.add("hidden"));
  $("page-" + name).classList.remove("hidden");
  document.querySelectorAll(".nav-btn").forEach(b => {
    b.classList.toggle("active", b.dataset.page === name);
  });
  $("message").classList.add("hidden");
  renderAll();
  refresh().then(renderAll).catch(() => {});
}

document.querySelectorAll(".nav-btn").forEach(btn => {
  btn.addEventListener("click", () => showPage(btn.dataset.page));
});

// ---------- Rendering ----------
function renderDashboard() {
  const total = books.reduce((sum, b) => sum + b.quantity, 0);
  const issued = activeIssues().length;
  $("stat-total").textContent = total;
  $("stat-issued").textContent = issued;
  $("stat-available").textContent = total - issued;

  const recent = books.slice(-5).reverse();
  $("recent-body").innerHTML = recent.length
    ? recent.map(b => `<tr><td>${esc(b.id)}</td><td>${esc(b.title)}</td><td>${esc(b.author)}</td><td>${esc(b.
category)}</td><td>${b.quantity}</td></tr>`).join("")
    : emptyRow(5, "No books yet.");
}

function renderBooks() {
  const q = $("search").value.trim().toLowerCase();
  const found = books.filter(b =>
    b.title.toLowerCase().includes(q) ||
    b.author.toLowerCase().includes(q) ||
    b.id.toLowerCase().includes(q)
  );
  $("books-body").innerHTML = found.length
    ? found.map(b => {
        const info = availabilityInfo(b);
        return `
      <tr>
        <td>${esc(b.id)}</td><td>${esc(b.title)}</td><td>${esc(b.author)}</td>
        <td>${esc(b.category)}</td><td class="num">${b.quantity}</td>
        <td><span class="avail ${info.cls}">${info.text}</span></td>
        <td class="actions">
          <button class="btn small edit" data-action="edit" data-id="${esc(b.id)}">✏ Edit</button>
          <button class="btn small danger" data-action="delete" data-id="${esc(b.id)}">■■ Delete</button>
        </td>
      </tr>`;
      }).join("")
    : emptyRow(7, q ? "No books match your search." : "No books yet. Add one above.");
}

function renderIssueForm() {
  const select = $("issue-book");
  const previous = select.value;
  const available = books.filter(b => availableCount(b) > 0);
  select.innerHTML = available.length
    ? available.map(b => `<option value="${esc(b.id)}">${esc(b.id)} - ${esc(b.title)} (${availabilityInfo(b).
text})</option>`).join("")
    : `<option value="">No books available</option>`;
  if (previous && available.some(b => b.id === previous)) select.value = previous;
}

// Uses the current title if the book still exists, otherwise the title saved with the record
function issueTitle(issue) {
  const book = findBook(issue.bookId);
  return esc(book ? book.title : issue.bookTitle || "(deleted book)");
}

function issuedRow(issue, withReturnButton) {
  const button = withReturnButton
    ? `<td class="actions"><button class="btn small success" data-action="return" data-id="${issue.
id}">Return</button></td>`
    : "";
  return `<tr><td>${esc(issue.bookId)}</td><td>${issueTitle(issue)}</td><td>${esc(issue.student)}</td><td>${
showDate(issue.issueDate)}</td><td>${showDate(issue.dueDate)}</td>${button}</tr>`;
```

```
}

function renderIssued() {
  const list = activeIssues();
  $("issued-body").innerHTML = list.length
    ? list.map(i => issuedRow(i, false)).join("")
    : emptyRow(5, "No books are currently issued.");
  $("return-body").innerHTML = list.length
    ? list.map(i => issuedRow(i, true)).join("")
    : emptyRow(6, "No books to return.");

  // Past records stay here even if the book was deleted later
  const history = issues.filter(i => i.returned).reverse();
  $("history-body").innerHTML = history.length
    ? history.map(i => `<tr><td>${esc(i.bookId)}</td><td>${issueTitle(i)}</td><td>${esc(i.student)}</td><td>${
showDate(i.issueDate)}</td><td>${showDate(i.returnDate)}</td></tr>`).join("")
    : emptyRow(5, "No returned books yet.");
}

function renderList() {
  $("list-body").innerHTML = books.length
    ? books.map(b => {
        const info = availabilityInfo(b);
        return `<tr><td>${esc(b.id)}</td><td>${esc(b.title)}</td><td>${esc(b.author)}</td><td>${esc(b.
category)}</td><td><span class="avail ${info.cls}">${info.text}</span></td><td><span class="badge ${info.
cls}">${info.status}</span></td></tr>`;
      }).join("")
    : emptyRow(6, "No books yet.");
}

function renderAll() {
  renderDashboard();
  renderBooks();
  renderIssueForm();
  renderIssued();
  renderList();
}

// ---------- Manage Books: add / edit / delete ----------
function resetBookForm() {
  editingId = null;
  $("book-form").reset();
  $("book-qty").value = 1;
  $("book-id").disabled = false;
  $("book-form-title").textContent = "Add a book";
  $("book-submit").textContent = "Add book";
  $("book-cancel").classList.add("hidden");
}

$("book-form").addEventListener("submit", async e => {
  e.preventDefault();
  const id = $("book-id").value.trim();
  const title = $("book-title").value.trim();
  const author = $("book-author").value.trim();
  const category = $("book-category").value.trim();
  const quantity = parseInt($("book-qty").value, 10);

  if (!id || !title || !author || !category) {
    return showMessage("Please fill in all fields.", "error");
  }
  if (!quantity || quantity < 1) {
    return showMessage("Quantity must be at least 1.", "error");
  }

  try {
    if (editingId) {
      const book = findBook(editingId);
      if (quantity < issuedCount(book.id)) {
        return showMessage(`Quantity cannot be lower than the ${issuedCount(book.id)} copies currently issued.
`, "error");
      }
      await dbPut("books", { ...book, title, author, category, quantity });
      showMessage("Book updated.", "success");
    } else {
      if (books.some(b => b.id.toLowerCase() === id.toLowerCase())) {
        return showMessage("That Book ID already exists. Use a different ID.", "error");
      }
      await dbPut("books", { id, title, author, category, quantity, addedAt: Date.now() });
      showMessage("Book added.", "success");
    }
    resetBookForm();
    await refresh();
    renderAll();
```

```
  } catch (err) {
    saveFailed(err);
  }
});

$("book-cancel").addEventListener("click", resetBookForm);
$("search").addEventListener("input", renderBooks);

$("books-body").addEventListener("click", async e => {
  const btn = e.target.closest("button");
  if (!btn) return;
  const book = findBook(btn.dataset.id);
  if (!book) return;

  if (btn.dataset.action === "edit") {
    editingId = book.id;
    $("book-id").value = book.id;
    $("book-id").disabled = true; // the ID cannot change
    $("book-title").value = book.title;
    $("book-author").value = book.author;
    $("book-category").value = book.category;
    $("book-qty").value = book.quantity;
    $("book-form-title").textContent = "Edit book";
    $("book-submit").textContent = "Save changes";
    $("book-cancel").classList.remove("hidden");
    $("book-title").focus();
    window.scrollTo({ top: 0, behavior: "smooth" });
  }

  if (btn.dataset.action === "delete") {
    if (issuedCount(book.id) > 0) {
      return showMessage("This book has copies issued. Return them before deleting.", "error");
    }
    if (!confirm(`Delete "${book.title}"? Its past issue and return records will be kept.`)) return;
    try {
      await dbDelete("books", book.id); // issue history stays in the "issues" store
      if (editingId === book.id) resetBookForm();
      await refresh();
      renderAll();
      showMessage("Book deleted.", "success");
    } catch (err) {
      saveFailed(err);
    }
  }
});

// ---------- Issue books ----------
$("issue-form").addEventListener("submit", async e => {
  e.preventDefault();
  const bookId = $("issue-book").value;
  const student = $("issue-student").value.trim();
  const issueDate = $("issue-date").value;
  const dueDate = $("issue-due").value;
  const book = findBook(bookId);

  if (!book) return showMessage("Please select a book.", "error");
  if (!student) return showMessage("Please enter the student's name.", "error");
  if (!issueDate || !dueDate) return showMessage("Please choose both dates.", "error");
  if (dueDate < issueDate) return showMessage("The due date cannot be before the issue date.", "error");
  if (availableCount(book) < 1) return showMessage("No copies of this book are available.", "error");

  try {
    await dbPut("issues", { id: Date.now(), bookId, bookTitle: book.title, student, issueDate, dueDate,
returned: false, returnDate: "" });
    $("issue-student").value = "";
    $("issue-date").value = today();
    $("issue-due").value = today(14);
    await refresh();
    renderAll();
    showMessage(`"${book.title}" issued to ${student}.`, "success");
  } catch (err) {
    saveFailed(err);
  }
});

// ---------- Return books ----------
$("return-body").addEventListener("click", async e => {
  const btn = e.target.closest("button");
  if (!btn || btn.dataset.action !== "return") return;
  const issue = issues.find(i => i.id === Number(btn.dataset.id));
  if (!issue) return;
  try {
    await dbPut("issues", { ...issue, returned: true, returnDate: today() });
```

```
    await refresh();
    renderAll();
    showMessage(`"${issue.bookTitle}" returned by ${issue.student}.`, "success");
  } catch (err) {
    saveFailed(err);
  }
});

// ---------- Due date follows the issue date (14 days later) ----------
function updateDueDate() {
  const issueDate = $("issue-date").value;
  if (issueDate) $("issue-due").value = addDays(issueDate, 14);
}
$("issue-date").addEventListener("input", updateDueDate);
$("issue-date").addEventListener("change", updateDueDate);

// ---------- Start ----------
async function start() {
  try {
    await openDatabase();
    await setUpData();
    await refresh();
  } catch (err) {
    console.error(err);
    showMessage("Could not open the browser database. Try another browser, or turn off private browsing.",
"error");
  }
  renderAll();
}

$("issue-date").value = today();
$("issue-due").value = today(14);
start();
```